

Movie Search Hub

Frontend Movie Discovery Application

A responsive, mobile-first web app with light and dark themes, powered by the OMDb API

Presentation & Defense Study Guide

Prepared by: Charles Musika

1. The Idea – What & Why

What problem does it solve?

There are millions of films, and finding information about one usually means opening a search engine, wading through adverts and clicking through several websites. People also forget the films they want to watch later. Movie Search Hub puts everything in one clean place: type a title, see matching movies with their posters, open a film to read its plot, cast, director and rating, and save the ones you like to a personal favourites list.

Why is this a useful idea?

Discovery should be fast and pleasant, especially on a phone, which is where most people look things up. The app answers the everyday question “what is that movie about, and is it any good?” in two taps, and remembers the user's favourites without requiring an account.

Motivation for choosing this project

- It is a real-world use of an API. Almost every modern application talks to an external service, so this project demonstrates that skill directly.
- It practises the core front-end skills: HTML structure, CSS design (including mobile-first responsive layout) and JavaScript logic.
- It needs no backend server or database, so it can be opened and demonstrated on any computer or phone with just a browser and an internet connection.
- Movie posters and ratings make it visually engaging, which makes it enjoyable to build and to present.

2. Technologies Used

The project uses only standard web technologies plus one external data service. There are no frameworks, no build tools, no backend and no database.

- **HTML5** – the structure of every page: navigation bar, search forms, result containers and footer.
- **CSS3** – all styling and layout. It is written mobile-first: the base styles are for phones, and min-width media queries add tablet and desktop layouts. CSS variables hold the colour theme (which is how the light and dark themes are switched), and CSS Grid arranges the movie cards.
- **JavaScript (ES6+)** – the behaviour: calling the API with `fetch()`, `async/await`, building the movie cards, handling the search form, and saving favourites.
- **OMDb API** – the external web service that supplies the movie data (explained fully in Section 3).
- **Browser localStorage** – lets the browser remember the user's favourites after the tab is closed, without a database.

3. Understanding the API – What, Why & How

What is an API?

API stands for Application Programming Interface. It is a set of rules that lets one program ask another program for data. A helpful comparison is a restaurant: the customer (our website) does not walk into the kitchen. Instead the customer gives an order to a waiter (the API), and the waiter brings back exactly what was ordered from the kitchen (the movie database). Our app never needs to store thousands of films itself; it simply asks the API.

What is the OMDb API?

OMDb (the Open Movie Database) is a free web service that provides information about films, series and episodes, including titles, years, posters, plots, casts, ratings and more. The app sends it a request over the internet, and it answers with the data in JSON format.

Why was OMDb chosen?

- It is free to use with a personal API key, which suits a student project.
- It is very simple: two request types cover everything the app needs (search by title and get details by ID).
- It returns clean JSON, which JavaScript can read directly.
- It supports cross-origin requests (CORS), so it can be called straight from the browser with no server of our own.
- It includes poster image links and IMDb ratings, which make the interface look professional.

What is an API key and why is it needed?

An API key is a short code (in this project a code of eight characters) that identifies who is making the requests. OMDb uses it to allow only registered users and to enforce the free-tier limit of roughly 1,000 requests per day. Without a valid key the API refuses the request. The key is stored in one place, the `API_KEY` constant at the top of `js/app.js`, so it can be changed easily.

How the app talks to the API

The app uses the two OMDb requests below. Both are simple HTTPS GET requests to `https://www.omdbapi.com/` with parameters added after a question mark.

Purpose	Request pattern	Used by
Search by title	?apikey=KEY&s=inception&type=movie	searchMovies() in app.js, used on the Search page. Returns a list of up to 10 matching movies.
Get full details	?apikey=KEY&i=tt1375666&plot=full	getMovie() in app.js, used on the Movie Details page and to refresh the home page posters.

Parameter	Meaning
apikey	The user's personal key that authorises the request.
s	Search text. Returns a list of movies whose titles match.
i	The IMDb ID of one specific movie (for example tt1375666 is Inception).
type	Restricts results to movies only (excludes series and episodes).
plot	Set to full to receive the long plot summary instead of the short one.

What the API sends back (example)

A search for “inception” returns JSON similar to this (shortened):

```
{
  "Search": [
    { "Title": "Inception",
      "Year": "2010",
      "imdbID": "tt1375666",
      "Type": "movie",
      "Poster": "https://m.media-amazon.com/images/..." },
    ...
  ],
  "totalResults": "48",
  "Response": "True"
}
```

When nothing matches, or the key is wrong, the API returns Response set to "False" together with an Error message instead of a Search list.

The step-by-step flow

- 1. The user types a title and presses Search.
- 2. search.js stops the normal form submit and calls searchMovies(query).
- 3. searchMovies() builds the URL, encodes the text safely with encodeURIComponent(), and calls fetch().
- 4. The browser sends the request over the internet; OMDb looks up the movies and replies with JSON.
- 5. The app converts the reply with response.json() and checks whether Response is "True".
- 6. If true, each movie is turned into a card by movieCard() and shown in the grid; if not, the “No movies found” message is displayed.
- 7. Clicking Details opens movie.html?id=IMDB_ID, where getMovie() requests the full information for that one film.

Key code (simplified)

```
async function searchMovies(query) {
  const res = await fetch(
    `https://www.omdbapi.com/?apikey=${API_KEY}&s=${encodeURIComponent(query)}&type=movie`
  );
  const data = await res.json();
  if (data.Response === "True") return data.Search;
  return []; // no results
}
```

The `async` and `await` keywords make JavaScript wait for the network reply without freezing the page, so the user still sees the “Searching...” spinner while the request is in progress.

Error handling and fallbacks

- If the network is down or the request fails, `search.js` catches the error and shows “Search could not be completed – check your internet connection or API key”.
- If a search has no matches, an empty-state message is shown instead of a blank page.
- If the API key is left as the placeholder, the app falls back to a small built-in demo catalogue so the pages can still be demonstrated.
- If a poster link is missing (the API returns “N/A”) or fails to load, a clapperboard placeholder is shown instead of a broken image.
- The home page shows its featured movies instantly from a built-in list (with working poster links), then quietly refreshes them with live data from the API. If the refresh fails, the built-in list stays on screen, so the page is never empty.
- Poster images are loaded with `referrerpolicy="no-referrer"` and `loading="lazy"`. The first stops image hosts from blocking requests that come from a local page, and the second loads posters only as they scroll into view, which saves data on phones.

Security and limitations to be aware of

- The key is visible in the browser because the project is frontend-only. This is acceptable for a free, low-risk key. In a production application the key would be hidden behind a small backend server that makes the calls on the user's behalf.
- Data is only as complete as OMDb's own records, and the free plan is limited to about 1,000 requests a day.
- Search returns the first 10 results only. The API supports paging through a page parameter, which is a natural future improvement.
- The app needs an internet connection for live search and for poster images.

A real bug we solved (good story for the defense)

At first every search answered “movie not found” even though a key had been entered. The cause was that the whole example URL from the OMDb email (<http://www.omdbapi.com/?i=...&apikey=41be50f1>) had been pasted into API_KEY instead of only the key. The code then built a request with a malformed apikey value, so OMDb rejected it and the app displayed “not found”. The fix was to store only the key itself. This showed how important it is to read the API's response (the Error field) when debugging, rather than assuming the code is wrong.

A second, smaller issue was found later: the poster links in the built-in demo list had expired and returned an error, so the home page showed placeholder icons instead of pictures. They were replaced with fresh links taken from the API, and the home page now also refreshes them live.

4. How the App Works (Page by Page)

Home page (index.html)

The landing page has a hero section with a large search box, quick-search chips (Inception, Interstellar, Avatar, The Matrix), a Featured Movies grid and an “Explore by Genre” section. The featured movies are shown instantly from a built-in list, then refreshed with live data from the API so the posters are always current. The navigation bar on every page also contains the light/dark theme button. Submitting the search box or pressing a chip sends the user to the Search page with the query in the address ([search.html?q=...](#)). This logic is in `js/home.js`.

Search page (search.html)

The user types a title and the app displays a grid of results with the poster, title, year and type. A spinner is shown while loading, a count such as “10 results” is displayed, and an empty-state message appears when nothing is found. The search text is kept in the URL so a result page can be refreshed or shared. This logic is in `js/search.js`.

Movie Details page (movie.html)

Opened from any card using the movie's IMDb ID. It shows the large poster, title, year, runtime, genre, IMDb rating, full plot, director, actors, language, country, release date and awards, plus an Add to / Remove from Favourites button. This logic is in `js/movie.js`.

Favourites page (favourites.html)

Shows all saved movies as cards. Pressing Saved removes a movie immediately, and an empty-state message with a “Find Movies” button appears when the list is empty. This logic is in `js/favourites.js`.

About page (about.html)

Describes the purpose, technology and responsiveness of the project, and explains how to connect an API key.

5. Key Features Explained Simply

1. Live movie search

Every search asks the OMDb API in real time, so the user can find almost any film, not just a fixed list.

2. Detailed movie information

A second, more detailed API request is made using the movie's unique IMDb ID, so the details page always shows accurate and complete information.

3. Favourites saved in the browser

When the user presses Save, the movie object is added to an array and stored with localStorage under the key movieHubFavourites, converted to text with JSON.stringify(). When any page loads, JSON.parse() turns it back into data. The count badge in the navigation bar updates automatically.

4. Reusable movie cards

One function, movieCard(), builds the HTML for a card and is reused on the Home, Search and Favourites pages. Any design change to a card is made in one place.

5. Security through escaping

All text coming from the API passes through escapeHTML() before it is inserted into the page. This prevents unexpected characters in a title from breaking the layout or injecting code (a basic protection against cross-site scripting).

6. Light and dark themes

A sun/moon button in the navigation bar switches between a dark cinematic theme and a clean light theme. The first time someone visits, the app follows their device setting (prefers-color-scheme); after that it remembers their choice in localStorage under the key movieHubTheme. The switch works by setting a data-theme attribute on the html element. The CSS defines every colour once as a variable (for example --bg, --surface, --text and --muted) and the light theme simply redefines those variables, so the whole interface changes instantly without duplicating any styles. A tiny script in each page's head sets the theme before the page is drawn, which prevents a flash of the wrong colours.

7. Mobile-first responsive interface

The layout begins as a two-column phone layout with large touch-friendly buttons and a collapsible menu, and expands to three and four columns with a full navigation bar on tablets and desktops.

6. Code Structure Overview (Beginner-Friendly)

The project separates structure (HTML), style (CSS) and behaviour (JavaScript). Each file has one clear job:

- **index.html, search.html, movie.html, favourites.html, about.html** – the five pages.
- **css/style.css** – the single stylesheet: colour variables for both themes, mobile-first base styles, and media queries at 560px, 800px and 1100px.
- **js/app.js** – the shared toolbox loaded on every page: the API_KEY, the demo catalogue, the favourites functions (getFavourites, saveFavourites, toggleFavourite), movieCard(), escapeHTML(), the API functions searchMovies() and getMovie(), the theme toggle (setupTheme) and the mobile menu.
- **js/home.js, js/search.js, js/movie.js, js/favourites.js** – one file per page, containing only the logic for that page.
- **README.md** – the project abstract and student details.

7. Design Decisions – Why It Looks and Works This Way

- **Mobile-first:** most people search for movies on a phone, so the base CSS is written for small screens and larger layouts are added with min-width media queries. This produces lighter, simpler code than shrinking a desktop design.
- **Dark cinematic theme (default for dark devices):** a dark background makes colourful posters stand out, mirrors the feel of a cinema, and is comfortable to look at in low light.
- **Light theme:** gives a clean, high-contrast option for bright environments and for users who prefer it. Text colours were chosen to keep good readability in both themes, and the gold rating colour is darkened in the light theme so it stays legible.
- **Theme that respects the user:** the app starts with the device preference, lets the user override it with one tap, and remembers the choice.
- **Purple-to-pink accent:** one gradient accent is used for buttons and highlights through CSS variables, keeping the whole site consistent.
- **Touch-friendly controls:** buttons and inputs are at least 44–48 pixels tall, and the search text is 16px so phones do not zoom in when the field is selected.
- **Consistent poster shape:** cards use a fixed 2:3 aspect ratio so the grid stays neat even when posters differ in size.
- **Accessibility:** visible focus outlines, labelled menu button, keyboard support (Escape closes the menu), and a reduced-motion setting for users who prefer less animation.
- **No backend:** using an API plus localStorage means no hosting costs, no database, and nothing to install.

8. Potential Presentation Questions & Answers

Q: What is an API and why did you use one?

A: An API lets one program request data from another. I used the OMDb API so the app can search a huge, up-to-date movie database without storing any movie data myself.

Q: Why did you choose OMDb instead of another movie API?

A: It is free with a personal key, simple to use (only two request types), returns clean JSON, supports browser requests without a server, and includes posters and IMDb ratings.

Q: What is an API key and where is it stored?

A: It is a personal code that identifies me to OMDb and allows the free daily limit. It is stored in the `API_KEY` constant at the top of `js/app.js`.

Q: Is it safe to keep the API key in frontend code?

A: Anyone can view it in the browser, so it is not truly secret. For a free, low-risk key this is acceptable in a student project. In production I would move the calls to a backend server that keeps the key private.

Q: How does the search actually work?

A: When the form is submitted, `searchMovies()` sends a fetch request to OMDb with the `s` parameter. The JSON reply is checked, and each movie in the Search list is turned into a card using `movieCard()`.

Q: What does `async/await` do?

A: Network requests take time. `async/await` lets the code wait for the reply without freezing the page, so the loading spinner can keep animating.

Q: What is JSON?

A: JavaScript Object Notation – a plain-text format for structured data. `response.json()` converts the API's text reply into a JavaScript object the code can use.

Q: What is the difference between the `s` and `i` parameters?

A: `s` searches by title text and returns a short list of matches. `i` requests one specific movie by its IMDb ID and returns full details.

Q: What happens if the API fails or there are no results?

A: If the request throws an error, a message asks the user to check their connection or key. If the API replies Response “False”, an empty-state message appears. The app never shows a blank screen.

Q: Why did every search say ‘movie not found’ at first?

A: The full example URL had been pasted into `API_KEY` instead of just the key, so OMDb rejected the request. Storing only the key fixed it. It taught me to check the API's actual response when debugging.

Q: Why did you use `localStorage` instead of a database?

A: The project is frontend-only. `localStorage` lets the browser remember favourites for free with no server. The trade-off is that data belongs to one browser on one device.

Q: What happens if the user clears their browser data?

A: The saved favourites would be lost because they only exist in that browser's localStorage.

Q: How did you make it mobile responsive?

A: I used a mobile-first approach: base CSS is for phones, and media queries at 560px, 800px and 1100px add more columns and a full navigation bar. Images use a fixed aspect ratio and grids use flexible columns.

Q: What is escapeHTML() for?

A: It converts special characters such as < and & in API data into safe text before inserting it into the page, protecting against broken layouts and cross-site scripting.

Q: Why does the app only show 10 results?

A: The OMDb search returns 10 results per page. Supporting more would require using its page parameter, which I list as a future improvement.

Q: Does the app work offline?

A: No. Live search and poster images need the internet. Saved favourites are stored locally but their posters are still loaded from the web.

Q: Does it support multiple users or accounts?

A: No. It is a single-user personal tool because favourites are tied to one browser.

Q: What was the hardest part?

A: Understanding how to read and validate the API's responses, and handling all the cases: success, no results, missing posters and network failure.

Q: What did you learn?

A: How to call and debug a real web API with fetch and async/await, how to design mobile-first with CSS Grid and media queries, and how to keep code organised with shared functions.

Q: What would you improve with more time?

A: Add pagination and filters (year, genre), move the API key to a small backend, add user accounts so favourites sync across devices, and add a trailer or recommendations section.

Q: How does the light/dark theme work?

A: All colours are defined as CSS variables on the root element. The light theme is a second set of values that applies when the html element has data-theme="light". JavaScript toggles that attribute when the user taps the sun/moon button, and stores the choice in localStorage. The first visit uses the device's own preference through the prefers-color-scheme media query.

Q: Why doesn't the page flash the wrong theme when it loads?

A: A very small script in the head of each page reads the saved theme and sets the attribute before the browser paints the page. Without it, the page would briefly appear dark and then switch to light.

Q: What if the poster images don't load?

A: Missing or broken posters are replaced by a clapperboard placeholder. The home page also has a built-in list with valid posters and refreshes it from the API, so it never appears empty.

Q: Could this be turned into a mobile app?

A: Yes. The API logic is separate from the interface, so the same OMDb calls could be reused in a framework such as React Native or wrapped as a Progressive Web App.

9. Summary / Closing Pitch

Movie Search Hub is a fast, professional, mobile-first web application with light and dark themes that connects to the OMDb API to let anyone search for films, read detailed information and keep a personal favourites list. It demonstrates the essential skills of modern front-end development: consuming a real-world API with fetch and async/await, handling errors gracefully, building a responsive interface with CSS Grid and media queries, and storing user data with localStorage – all without a backend or database.

Charles Musika